Society App: Technical Design Document

1. Links

- GitHub: https://github.com/somaxcodes/society-app
- Live (Render): https://society-app-4t15.onrender.com

2. Architecture and why

The stack

- **Web framework:** FastAPI. Pages are rendered on the server with Jinja2 templates. There is no separate frontend app and no JavaScript framework.
- **Database:** SQLite, accessed through SQLAlchemy 2.x. The whole database is one file, `society.db`.
- **Styling:** Pico.css (loaded from a CDN in `base.html`) plus one custom stylesheet, `app/static/style.css`. A small `app.js` handles the filter dropdowns and the highlight when you jump to a linked complaint.
- **Auth:** email and password. Passwords are hashed with bcrypt, and a signed session cookie (Starlette `SessionMiddleware`) keeps you logged in. The app refuses to start without a `SECRET_KEY` in the environment.
- **AI:** the Groq API, using the `openai/gpt-oss-20b` model.
- **Deploy:** Render free tier, deployed from the GitHub repo.
- **Tests:** pytest. At the time of writing there are 259 tests.

Why this stack

I wanted one person to finish a working, deployed app in about a week, so I picked tools with very little setup. SQLite needs no database server. Jinja2 means no separate API layer and no frontend build step. Pico.css gives a readable, mobile-friendly base without a design system of my own. FastAPI and SQLAlchemy are tools I could move quickly in.

The cost is real, though. SQLite on Render's free tier loses its data when the service restarts, so the app seeds itself on startup when the database is empty. A real deployment would use a hosted database.

3. Data model, main flows, and how the AI part works

Data model

Defined in `app/models.py`.

- **User:** email (unique), bcrypt-hashed password, `role` ("resident" or "staff"), and an optional `flat_id`. Staff have no flat.
- **Flat:** flat number (unique, like `T07-1203`), owner name, area in sqft, and the owner's UPI ID. The part before the dash is treated as the tower.
- **Bill:** belongs to a flat. Holds `amount_due`, `amount_paid` (starts at 0) and `due_date`. In the seed data, `amount_due` is area times ₹6.50.
- **BankPayment:** one row for every line in an uploaded bank statement. It stores the date, amount, reference, UPI ID, the matched flat if there is one, a `status` ("matched", "needs_review" or "duplicate") and a plain-English `reason`. A database check constraint limits the status values.
- **BankUploadLog:** one row per successful upload, with the filename and time. The Bank Upload page shows the latest one.
- **Complaint:** flat, description, category, urgency, status, `created_at`, `resolved_at`, `is_safety_flagged`, and `linked_to_id`. When a complaint is a duplicate, `linked_to_id` points at the original. Check

constraints limit category, urgency and status.

Flow 1: bill payment

- A resident logs in and sees their flat's bill: amount due, amount paid, outstanding balance, days overdue, and their open complaints.
- They enter an amount and submit.
- The server rejects zero, negative, or anything above the outstanding balance.
- Otherwise it adds the amount to `amount_paid` and redirects back with a "payment received" banner.

There is no real payment gateway. The payment is only recorded.

Flow 2: bank statement matching

Staff upload a CSV with the columns `date, amount, reference, upi_id`, in that order.

Validation - Every row is parsed and checked before anything is saved. - One bad amount or date rejects the whole file, with a message naming the line. - That way a statement is never left half imported.

Matching (`match_payment()` in `app/matching.py`, which never touches the database) - A reference already seen, in an earlier upload or earlier in the same file, makes the row a duplicate. - Otherwise it looks for a flat number inside the reference text. Dashes and spaces are ignored, so "T07 1203" matches "T07-1203". - It also looks for a UPI ID that matches a flat exactly. - If the flat number and UPI ID agree, or only the UPI ID matches, that flat is the candidate. - If they point at different flats, or only the flat number matched, the row goes to review. - An amount that happens to match what some flat owes is used only to explain a review reason. It never matches a row by itself. - If the candidate flat is already fully paid, or the payment is more than it owes, the row goes to review. - Otherwise the row is matched and the bill's `amount_paid` goes up immediately.

Staff pages - The upload result page shows the counts and the rows needing review. - The Review Queue lists every `needs_review` row, with a best-guess flat shown for reference only. - The Processed Transactions page lists every matched and duplicate row.

Flow 3: complaint filing with AI classification

- A resident types a description. The server trims it and checks that it is 1 to 1000 characters.
- The complaint is classified, checked for duplicates, saved as "Open", and the resident lands on their list with a confirmation banner.
- Staff see every complaint sorted by urgency (High first, newest first within a level) and can filter by category and status.
- Staff can change a complaint's category and status.

How the AI part works

This lives in `app/classification.py`, `app/safety_check.py` and `app/main.py`. It runs in six steps.

1. The safety check runs first, with no AI. - `check_safety_flag()` is plain keyword matching. - It handles common misspellings, text broken up by spaces or punctuation (like "g.as le-ak"), and a few hand-picked word stems. - Some words count on their own: fire, smoke, shock, collapse, trapped, injured. - Some count only in pairs: lift with stuck, gas with leak or smell, wire with spark, live with wire. - A structural rule pairs a danger word like "cracking" or "wobbly" with a structural noun like "ceiling" or "railing". "My chair is wobbling" is not flagged. - Short words like "fir" match only as whole words, so "first floor" is not flagged. - The check is literal on purpose. Section 5 covers the tradeoff.

2. Groq classification. - One Groq call at temperature 0 returns JSON with a `category` and an `urgency`. - The prompt lists the allowed categories and defines Low, Medium and High with examples. - It tells the model to judge the real situation, not how urgent the message sounds.

3. Fallback. - If the call fails (bad key, timeout, invalid JSON, retired model), the category becomes "other" and the urgency "Medium". - If the call works but a value isn't allowed, such as "banana", only that field falls back. A valid urgency survives a nonsense category. - A broken AI never blocks a complaint from being filed.

4. The safety override. - If the keyword check fired, urgency is forced to High. - This runs last, so it beats both the AI's answer and the fallback.

5. Duplicate linking. `find_duplicate()` narrows candidates without any AI call first. - A candidate must share the category and tower, still be Open or In Progress, and have been filed within the last 7 days. - Groq is then asked, once per candidate and oldest first, whether the two complaints describe the same underlying problem. - A "yes" links the new complaint to that candidate. - Any error returns "no duplicate", so a broken duplicate check never blocks filing either. - Links always point at the oldest complaint, so a cluster is a star shape and "everything about this issue" is one simple query.

6. Cluster escalation. When a new complaint is linked, `escalate_duplicate_cluster_urgency()` takes over. - It finds the true root of the cluster and points the new complaint at it. - It then sets every complaint in the cluster to the highest urgency among them. - If any one of them is safety-flagged, all of them are. - Urgency only goes up this way, never down.

What the pages show - A red banner for `is_safety_flagged`. - An amber "High priority" banner when urgency is High but no safety keyword matched. That means the AI rated it High, either directly or through a linked duplicate, without a keyword confirming it.

4. UI and design decisions

- **Separate staff and resident views.** Each role gets its own navigation and pages. Residents see only their own flat's bill and complaints, and staff see every flat plus the bank tools. `require_role()` returns 401 if you aren't logged in and 403 if you have the wrong role. The brief only defined these two roles, so there is no committee role. The "committee dashboard" is simply the staff dashboard.
- **Color-coded badges and banners.** Urgency and status appear as badges with the text always visible, so color is never the only signal. Safety gets its own red banner instead of reusing "High", because the two come from different places: the banner from the keyword check, urgency from the AI or the escalation rules. The amber banner lets staff tell "a keyword confirmed danger" from "only the AI rated it High".
- **Pico.css, not a custom framework.** It is readable and responsive out of the box, needs no build step, and keeps the markup as plain semantic HTML. My own CSS on top handles the palette, cards, badges and hover states. `docs/design-brief.md` asked for large readable text, good contrast, big buttons and clear labels, since some users may be elderly or not at ease with technology. Pico made those basics easier to get right than starting from nothing.
- **Plain error messages.** The login page shows the same error for an unknown email and a wrong password, so it can't be used to find out which emails have accounts.

5. What was cut and what's next

Cut or skipped

- **Stage 4 (responsive breakpoints) was skipped.** This is a server-rendered internal tool, and tuning breakpoints properly would have cost hours for little benefit, so it went down as a known limitation. A small mobile pass came later (one max-width: 480px block). I have since checked it on an actual phone and it works fine.
- **Stage 5 (a separate regression pass) was folded into pytest.** Tests covering login and roles, bill payment, complaint submission, duplicate linking, urgency escalation and bank CSV matching seemed a more honest check than clicking through pages. The AI calls are replaced with fakes in the tests. `test_urgency_manual.py` calls the real AI and is meant to be read by a person, not asserted.
- **OTP login was not built.** The brief asked for it, but email and password with hashed passwords and a signed cookie is simpler and secure enough for a demo.

- **No committee role.** See section 4.

Known limitations

- **One static bill per flat.** The brief's sample bank statement covers several months, but the app creates one `Bill` per flat at seed time and has no monthly billing cycle. That is why the Review Queue shows so many "possible overpayment" flags: any payment past the single fixed amount looks like an overpayment. A real system would create a bill each cycle, match payments against the open one, and credit forward or flag leftovers.
- **Free-tier storage resets by design, not oversight.** Render's free tier has no persistent disk, so SQLite resets on every restart or period of inactivity. This is treated as a known constraint: the app auto-seeds realistic demo data whenever it starts with an empty database, which on Render is every restart, so the live deployment always shows a working, populated state instead of an empty one. The tradeoff is that anything entered live (a new complaint, a bank upload) only persists until the next restart. That is acceptable for a demo review, but a production deployment would need a persistent database.
- **The safety check is literal on purpose.** "Fire exit door is squeaky" gets flagged like a real fire. A wrong flag costs a staff member a few seconds, while a missed gas leak could cause real harm, so the check leans toward over-flagging.
- **Duplicate-complaint privacy tradeoff.** When two complaints are linked, each resident can see the other's description on their own complaints page. I accepted that because complaints are about shared common-area problems, not private information. The catch is that a resident who puts something private in a complaint could have it shown to a neighbor in the same tower.
- **Duplicate prevention for bank rows lives in application code,** not in a database constraint. The unique constraint on `BankPayment.reference` was removed so duplicate rows can still be stored as an audit trail. The check is a plain match on the reference string, and date and amount are ignored.

Next

`decisions.md` has no TODO list, so these come from the limitations above and from reading the code.

- Generate a bill each billing cycle and match payments against the open one.
- Move off SQLite on Render's free tier, or add a persistent disk or hosted database.
- Add real database migrations. `create_all` creates missing tables but doesn't change existing ones, so a new table or column needs a manual step on an existing database.
- Finish the full responsive work that Stage 4 skipped.